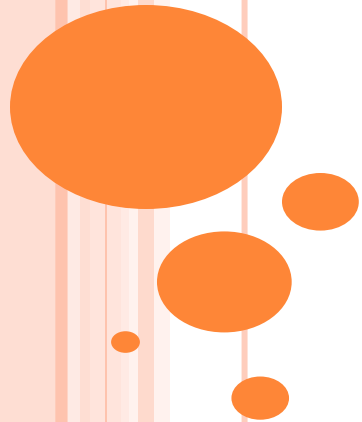


SECURIZACIÓN DE MICROSERVICIOS MEDIANTE TOKEN



Token vs autenticación básica

➤ Un token es una cadena de caracteres codificada que incluye el identificador de usuario, rol y otros datos adicionales:

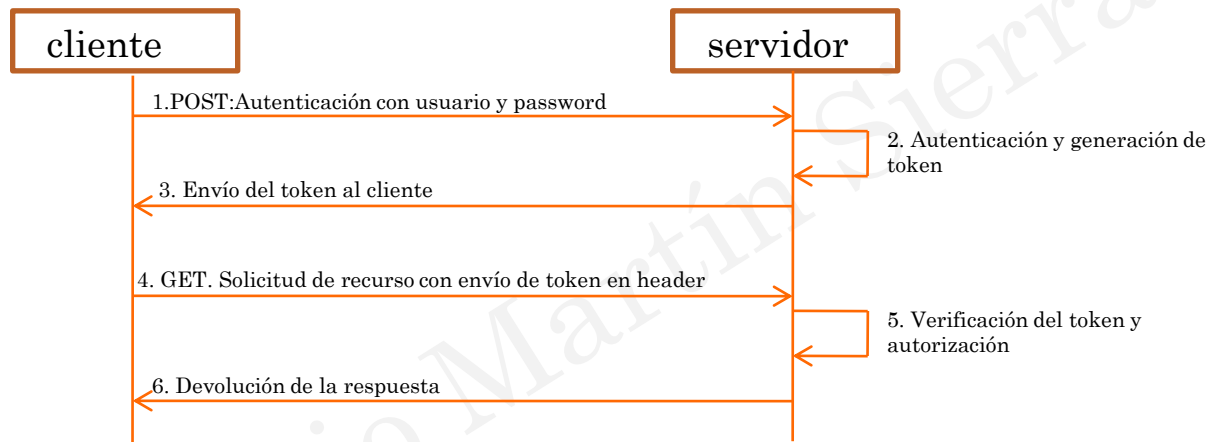
38dfa4a229e002efd11.\$2ee44opgm

➤ Mientras que con la autenticación básica se deben enviar siempre las credenciales para autenticar y autorizar al usuario, utilizando tokens la autenticación se realiza una sola vez para generar el token, el resto de peticiones envían el token generado en el que se incluye toda la información del cliente, a excepción de la contraseña

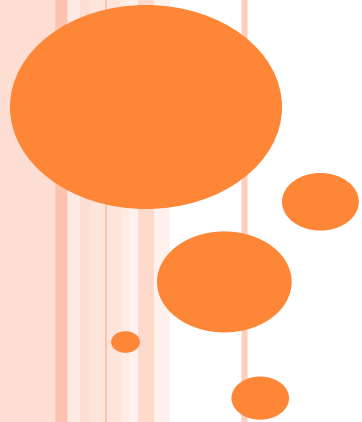
➤ El token es más seguro, dado que no incluye contraseña.



Funcionamiento



TOKEN JWT



¿Qué es un token JWT?

- Mecanismo para comunicar de forma segura un microservicio con una aplicación cliente y proceder a su identificación.
- Se basa en el uso de una cadena JSON codificada y firmada, que incluye los datos del cliente (usuario, roles,...).
- El cliente envía el token JWT al microservicio en la cabecera de las peticiones, éste la decodifica y verifica que no ha sido alterada en tránsito, extrayendo a continuación la información del cliente para proceder a su autorización.

petición
HTTP

cabecera

cuerpo

authorization=token_jwt	
-------------------------	--

Estructura de un token JWT

➤ Un token JWT es una cadena codificada que consta de tres partes separadas por un ".":

header . payload . signature

```
header {
  {
    "alg": "HS256",
    "typ": "JWT"
  }
}

payload {
  {
    "iat": 1632173019,
    "sub": "admin",
    "authorities": [
      "ROLE_ADMIN",
      "ROLE_USER"
    ],
    "exp": 1632259419
  }
}

signature {
  HMACSHA256(
    base64UrlEncode(header) + "." +
    base64UrlEncode(payload),
    secret_key
  )
}
```

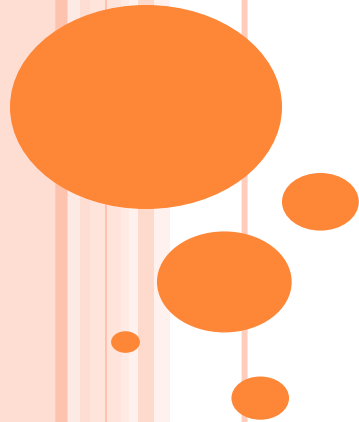
header: Incluye dos campos, el tipo de token (siempre JWT) y el mecanismo de firma

payload: Contiene la información del cliente, como el nombre, roles, etc. También el tiempo de vida del token

signature. Se construye utilizando la función especificada en la cabecera y que recibe como parámetro la cabecera más el payload codificado, además de la clave secreta preestablecida. Esto permite verificar la integridad del mensaje

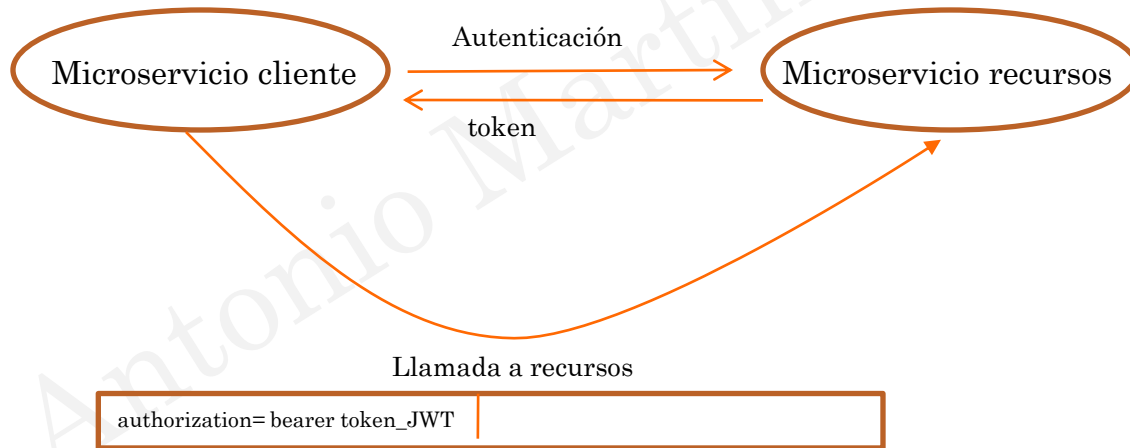
eyJhbGciOiJIUzUxMiJ9
.eyJpYXQiOiE2MzQxNDI0ODgsInN1YiI6ImFkbWluIiwiaXV0aG9yaXRpZXMiOiUk9MRV9BRE1JTilIsIlJPTEVfVVNFUiJdLCJleHAiOiE2MzQyMjg4ODh9
.JsiPFdtOjkCW5AwwmloE3cz_WTl3ZM9CC4_ZKw7XbBH-zEedc0-geuYEc3BIWM3Yf7dtZQA3c5mmToRizIjk6g

SECURIZACIÓN DE MICROSERVICIOS MEDIANTE TOKEN JWT

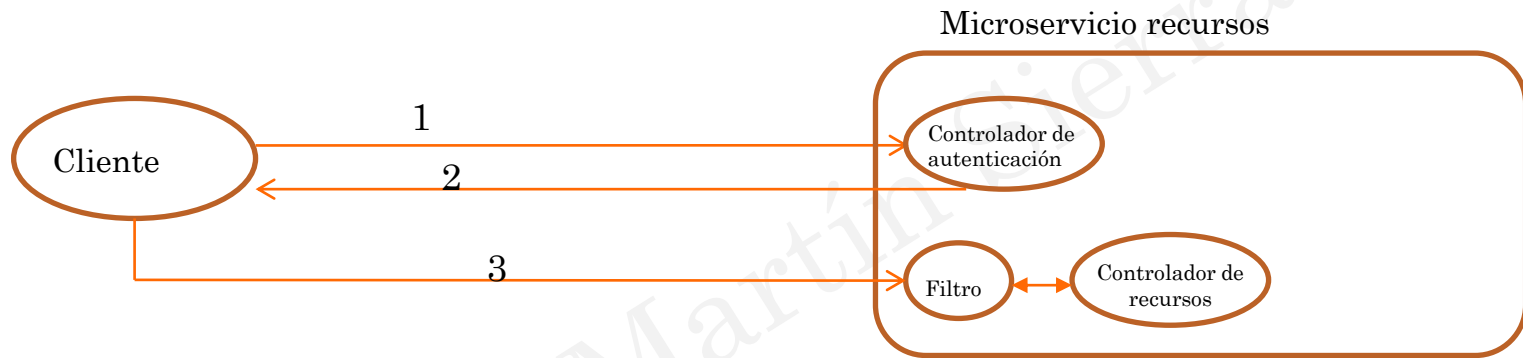


Token JWT y microservicios

- El microservicio cliente se autentica previamente en el microservicio que quiere utilizar para obtener el token.
- El cliente utiliza el token para acceder a los recursos



Microservicio de recursos



1.-El cliente se autentica a través de un controlador específico

2.-El controlador genera el token y se lo envía al cliente

3.-El cliente presenta el token en cada petición y un filtro se encarga de su validación

Dependencias

➤ Además del starter de spring security, se requiere la librería jjwt encargada de generar y decodificar el token:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.1</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.1</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.1</version>
  <scope>runtime</scope>
</dependency>
```



Autenticación y generación del token

➤ Se deberá implementar un controller específico encargado de la autenticación y generación del token

```
@RestController
public class AuthController {
    AuthenticationManager authManager;
    public AuthController(AuthenticationManager authManager) {
        this.authManager=authManager;
    }
    @PostMapping("login")
    public ResponseEntity<String> login(@RequestParam("user") String user, @RequestParam("pwd") String pwd) {
        try {
            Authentication authentication=authManager.authenticate(new UsernamePasswordAuthenticationToken(user,pwd));
            return new ResponseEntity<>(getToken(authentication),HttpStatus.OK);
        }catch(AuthenticationException ex) {
            ex.printStackTrace();
            //si el usuario no es válido, enviamos el código de estado 401
            return new ResponseEntity<>("No autorizado",HttpStatus.UNAUTHORIZED);
        }
    }
    private String getToken(Authentication authentication) {
        //generación del token a partir de los credenciales
    }
}
```

Configuración de seguridad

➤ La configuración de spring security permanece intacta, únicamente habrá que indicar el filtro encargado de verificar el token:

```
@EnableWebSecurity
@Configuration
public class SecurityConfig {
    AuthenticationManager auth;
    @Bean
    public AuthenticationManager authManager(AuthenticationConfiguration conf) throws Exception{
        auth=conf.getAuthenticationManager();
        return auth;
    }
    @Bean
    public JdbcUserDetailsManager usersDetailsJdbc() {
        :
    }
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception{
        http.csrf(cus->cus.disable())
        .authorizeHttpRequests(auth->
            aut.requestMatchers(HttpMethod.POST,"/contactos").hasRole("ADMIN")
            .requestMatchers(HttpMethod.DELETE,"/contactos/*").hasAnyRole("ADMIN","OPERATOR")
            .requestMatchers("/contactos").authenticated()
            .anyRequest().permitAll()
        )
        .addFilter(new JWTAuthorizationFilter(auth));
        return http.build();
    }
}
```

Filtro de seguridad



Filtro de seguridad

➤ **Subclase de BasicAuthenticationFilter. En doInternalFilter() se decodifica el token y se obtiene la información del usuario, pasando el control a Spring Security para que proceda a la autorización:**

```
public class JWTAuthorizationFilter extends BasicAuthenticationFilter {
    public JWTAuthorizationFilter(AuthenticationManager authManager) {
        super(authManager);
    }
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        String header = req.getHeader(ENCABEZADO);
        if (header == null || !header.startsWith(PREFIJO_TOKEN)) {
            chain.doFilter(req, res);
            return;
        }
        //obtenemos los datos del usuario a partir del token
        UsernamePasswordAuthenticationToken authentication = getAuthentication(req);
        //la información del usuario se almacena en el contexto de seguridad
        //para que pueda ser utilizada por Spring security durante el
        //proceso de autorización
        SecurityContextHolder.getContext().setAuthentication(authentication);
        chain.doFilter(req, res);
    }
    private UsernamePasswordAuthenticationToken getAuthentication(HttpServletRequest request) {
        //obtención de datos del usuario a partir del token
    }
}
```

